PromptKit

Composable, version-controlled prompts
for AI-assisted software engineering

aka.ms/PromptKit · MIT License · v0.2.0

Engineering Deep-Dive · Getting Started

The Problem

Ad-hoc prompts
Written once, pasted everywhere, never improved

No version control
Can't track what changed, who changed it, or why

No reuse
Every engineer reinvents the same persona / guardrails / format

No testing
No way to verify prompt quality or catch regressions

Inconsistent outputs
Same task → wildly different results across the team

The Fix: Treat Prompts as Code

Modular

Compose from reusable building blocks

Version-Controlled

Track changes via Git like any code

Testable

Reference-compare against known-good

Extensible

Add your own personas, protocols, templates

PromptKit applies the same engineering discipline you use for software to the prompts that build it.

Architecture: The 5-Layer Composition Stack

① Persona — Who the LLM becomes

systems-engineer, security-auditor, devops-engineer ...

② Protocols — How it reasons + guardrails

anti-hallucination, root-cause-analysis, thread-safety ...

③ Format — Output structure & rules

requirements-doc, investigation-report, pipeline-spec ...

④ Taxonomy — Domain classification (optional)

severity levels, priority tiers, risk categories

⑤ Template — The task with parameters

investigate-bug, review-code, author-design-doc ...

Templates declare which components to compose via YAML frontmatter → bootstrap engine snaps them together

What's in the Box

10 Personas

systems-engineer, security-auditor

software-architect, devops-engineer

reverse-engineer, specification-analyst

workflow-arbiter, implementation-engineer

test-engineer, promptkit-contributor

20 Protocols

Guardrails (3): anti-hallucination, self-verification, operational-constraints

Analysis (4): memory-safety-c, memory-safety-rust, thread-safety, security-vulnerability

Reasoning (13): root-cause-analysis, traceability-audit, code-compliance-audit, requirements-elicitation … and 9 more

11 Formats

requirements-doc, design-doc, validation-plan, investigation-report

implementation-plan, agent-instructions, pipeline-spec, release-notes

multi-artifact, triage-report, promptkit-pull-request

27 Task Templates: investigate-bug · review-code · author-requirements-doc · author-design-doc · audit-traceability · audit-code-compliance · plan-implementation · author-pipeline · triage-issues · extend-library … and more

Quick Start — 3 Commands

Interactive mode

```
npx @alan-jowett/promptkit
```

Detects your LLM CLI (Copilot / Claude) and launches an interactive prompt-building session

Browse templates

```
npx @alan-jowett/promptkit list
```

Lists all available templates with descriptions
Add --json for machine-readable output

Assemble a prompt

```
npx @alan-jowett/promptkit assemble investigate-bug \
  -p problem_description="Segfault on startup" \
  -o investigation.md
```

Composes persona + protocols + format + template
into a single ready-to-use prompt file

Prerequisites: Node.js 18+ · Optional: GitHub Copilot CLI or Claude Code for interactive mode

How Assembly Works

Persona

```
# Identity
You are a senior
systems engineer …
```

→

Protocols

```
# Reasoning
## Anti-
Hallucination
## Root-Cause …
```

→

Format

```
# Output Format
Investigation Report
with sections …
```

→

Template

```
# Task
Investigate the bug:
{{problem_descriptio
n}}
```

Output: Single composed prompt file — ready to paste into any LLM
or use as agent instructions (.github/instructions/*.md, CLAUDE.md, .cursorrules)

Pipeline Chaining: Multi-Stage Workflows

Each template declares input/output contracts.

One template's artifact becomes the next template's input.

Build complete document lifecycles without copy-paste.

Get Started Today

Try it now

`npx @alan-jowett/promptkit`

Zero install — just run it. Walks you through template selection and prompt assembly.

Browse the repo

`aka.ms/PromptKit`

Read the templates, protocols, personas. Understand the composition model.

Contribute

`Use the extend-library template`

Add your own personas, protocols, or templates. PromptKit eats its own dog food — the contribution workflow is itself a PromptKit template.

Share feedback

`File issues or start discussions`

What templates are missing? What workflows do you wish PromptKit supported?